ЛАБОРАТОРНАЯ РАБОТА №2. ВЕРСТКА (ЧАСТЬ 1)

Цель: изучить основы верстки в Flutter, создать и оформить первый экран приложения.

Задачи:

- подготовить проект для верстки экранов;
- ознакомиться с основными виджетами в Flutter;
- создать и сверстать первый экран приложения, содержащий какой-либо список данных. Дизайн может выглядеть следующим образом:



- написать отчет о проделанной работе.

ПОЯСНЕНИЯ К ВЫПОЛНЕНИЮ ЗАДАЧ

Установка библиотек

– go_router для навигации;
– get_it для инъекции зависимостей;
– talker и talker_flutter – логгер для отслеживания работы приложения;

Для установки можно использовать либо файл конфигурации pubspec.yaml, либо терминал IDE. Все подробности есть на официальных сайтах:

– https://pub.dev/packages/go_router;
– https://pub.dev/packages/get_it/install;
– https://pub.dev/packages/talker/install;
– https://pub.dev/packages/talker_flutter/install.

Зависимость (dependency) – объект, который нужен другому классу для работы.

Инъекция зависимостей (Dependency Injection, DI) – шаблон проектирования, который используется для уменьшения жесткой связанности кода. Он позволяет передавать зависимости объекта через конструктор, метод или свойство вместо создания зависимости внутри самого объекта.

Все зависимости будут регистрироваться в файле di.dart внутри директории lib/di/:



Для начала здесь можно зарегистрировать экземпляр логгера:

di.dart

```
final getIt = GetIt.instance;
final talker = TalkerFlutter.init();

Future<void> setupLocator() async {
  getIt.registerSingleton(talker);
}
```

Подготовка проекта к верстке

Далее – верстка – часть организации слоя приложения, за который будет отвечать директория lib/app/ (lib/application/) со следующей структурой:

– extensions хранит вспомогательные функции;

– features содержит файлы экранов и их логики;

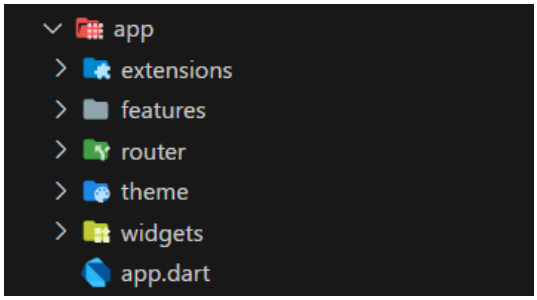
– router отвечает за навигацию внутри приложения;

– theme – папка с настройками темы;

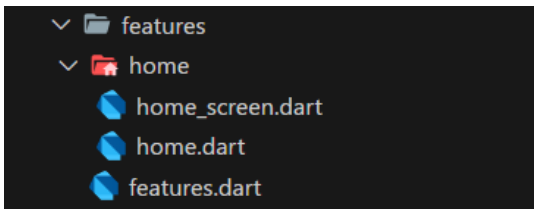
– widgets содержит виджеты приложения.

Файл app.dart понадобится для экспорта содержимого всей папки, чтобы оно было доступно из других частей приложения.

В дальнейшем почти все файлы, имена которых будут совпадать с именем папки, послужат для экспорта, и не будут комментироваться.



Предварительно нужно будет создать заготовки будущих экранов, чтобы при подготовке проекта к их реализации не возникало ошибок неопознанных имен. Файловая структура одного такого экрана может выглядеть вот так:



home_screen.dart

```
class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  @override
  Widget build(BuildContext context) {
    return Container();
  }
}
```

home.dart

```
export 'home_screen.dart';
```

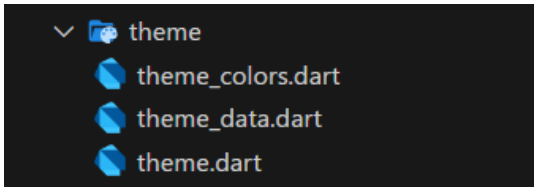
features.dart

```
export 'home/home.dart';
```

Первое, с чего можно начать подготовку к верстке, – задать будущему приложению тему. Для минимальных настроек понадобятся всего два файла:

– theme_colors.dart будет определять цветовую палитру приложения;

– theme_data.dart обеспечит основную конфигурацию темы.



theme_colors.dart

```
abstract final class ThemeColors {
  static const Color accent = Color.fromRGBO(20, 52, 78, 1);
  static const Color white = Color.fromRGBO(255, 255, 255, 1);
  static const Color black = Color.fromRGBO(17, 17, 17, 1);
  static const Color transparent = Color.fromRGBO(0, 0, 0, 0);
}
```

theme_data.dart

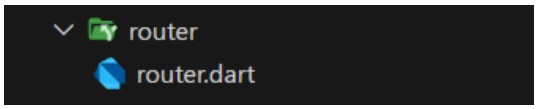
```
class AppTheme {
  static ThemeData get lightTheme {
    return ThemeData(
```

```
      useMaterial3: true,
      scaffoldBackgroundColor: ThemeColors.white,
      colorScheme: _colorScheme,
      appBarTheme: _appBarTheme,
      // таким же образом сюда можно добавить и темы других виджетов
    );
  }
}

final _colorScheme = ColorScheme.fromSeed(
  seedColor: ThemeColors.accent,
);

final _appBarTheme = AppBarTheme(
  toolbarHeight: 50,
  elevation: 0,
  scrolledUnderElevation: 0,
  centerTitle: true,
  titleTextStyle: TextStyle(
    fontSize: 20,
    fontWeight: FontWeight.bold,
    color: ThemeColors.white,
  ),
  backgroundColor: ThemeColors.black,
);
```

Следующий шаг – настройка маршрутизации. Для этого будет достаточно файла router.dart:



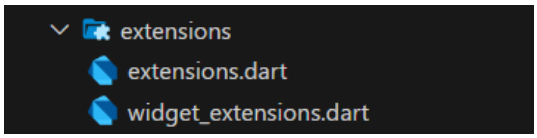
router.dart

```
final _rootNavigationKey = GlobalKey<NavigatorState>(debugLabel: 'root');

final router = GoRouter(
  observers: [TalkerRouteObserver(talker)],
  debugLogDiagnostics: true,
  initialLocation: '/home',
  navigatorKey: _rootNavigationKey,
  routes: [
    GoRoute(
      path: '/home',
      pageBuilder: (_, state) => MaterialPage(
        key: state.pageKey,
        child: const HomeScreen(),
      ),
    ),
    // для следующей лабораторной работы
    // GoRoute(
    //   path: '/content/:id',
    //   pageBuilder: (_, state) => MaterialPage(
    //     key: state.pageKey,
```

```
    //     child: ContentScreen(contentId: state.pathParameters['id']!),
    //   ),
    // ),
  ],
);
```

Напоследок, в папке extensions можно создать некоторые вспомогательные функции для верстки виджетов.



widget_extensions.dart

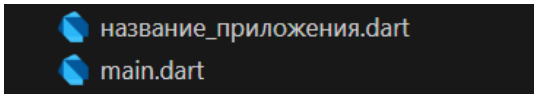
```
extension EmptyPadding on num {
  SizedBox get ph => SizedBox(height: toDouble());
  SizedBox get pw => SizedBox(width: toDouble());
}
```

К концу подготовки файл экспорта папки app должен выглядеть следующим образом:

app.dart

```
export 'extensions/extensions.dart';
export 'features/features.dart';
export 'router/router.dart';
export 'theme/theme.dart';
export 'widgets/widgets.dart';
```

Последнее, что остается сделать, – настроить класс приложения и некоторые параметры его запуска. Для этого нужно будет вынести его из main.dart в новый файл директории lib/:



main.dart

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  setupLocator();

  FlutterError.onError = (details) {
    return talker.handle(details.exception, details.stack);
  };

  runApp(AppName());
}
```

название_приложения.dart

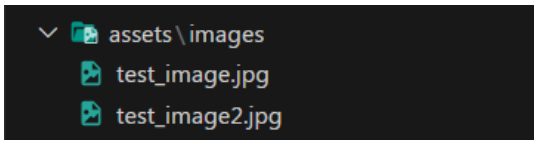
```
class AppName extends StatelessWidget {
  const AppName({super.key});

  @override
```

```
  Widget build(BuildContext context) {
   return MaterialApp.router(
    debugShowCheckedModeBanner: false,
    title: 'Название приложения',
    theme: AppTheme.lightTheme,
    routeInformationProvider: router.routeInformationProvider,
    routeInformationParser: router.routeInformationParser,
    routerDelegate: router.routerDelegate,
   );
  }
}
```

Теперь можно приступить к верстке. В примере экран будет состоять из списка элементов, каждый из которых содержит название, описание и изображение.

Пару тестовых изображений можно добавить в проект заранее, создав в глобальной папке проекта папку assets:



Чтобы приложение могло их видеть, путь к этой папке нужно указать в файле конфигурации:

pubspec.yaml

```
 assets:
  - assets/images/
```

Если в приложении присутствуют повторяющиеся виджеты, их следует выносить в отдельные классы и переиспользовать по мере необходимости. Здесь, например, можно вынести карточку элемента списка.

Для этого понадобится отдельная директория, где будут собраны пользовательские виджеты. Если заранее известно, что виджет будет использоваться только на одном экране, директорию можно создать внутри его папки, иначе – в глобальной папке уровня приложения.

content_card.dart

```
class ContentCard extends StatelessWidget {
 const ContentCard({super.key});

 @override
 Widget build(BuildContext context) {
  final imageSize = 100.0;

  return InkWell(
   // onTap: () => context.push('/content/${content.id}'),
   borderRadius: BorderRadius.circular(16),
   child: SizedBox(
    height: imageSize,
    child: Row(
     spacing: 16,
     crossAxisAlignment: CrossAxisAlignment.start,
     children: [
```

```
        ClipRRect(
          borderRadius: BorderRadius.circular(16),
          child: Image.asset(
            'assets/images/test_image.jpg',
            height: imageSize,
            width: imageSize,
            fit: BoxFit.cover,
          ),
        ),
        Expanded(
          child: Column(
            spacing: 4,
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                'Title',
                maxLines: 1,
                overflow: TextOverflow.ellipsis,
                style: Theme.of(context).textTheme.titleLarge,
              ),
              Expanded(
                child: Text(
                  'Description',
                  maxLines: 3,
                  overflow: TextOverflow.ellipsis,
                  style: Theme.of(context).textTheme.bodyMedium,
                ),
              ),
            ],
          ),
        ),
      ],
    ),
  ),
 );
 }
}
```

Теперь на самом экране данный виджет можно использовать следующим образом:

home_screen.dart

```
class HomeScreen extends StatefulWidget {
 const HomeScreen({super.key});

 @override
 State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
 @override
 Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: Text('Home')),
    body: SingleChildScrollView(
      padding: EdgeInsets.all(16),
```

```
      child: Column(
        spacing: 20,
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            'Header',
            style: Theme.of(context).textTheme.headlineLarge,
          ),
          ListView.separated(
            primary: false,
            shrinkWrap: true,
            itemCount: 10,
            itemBuilder: (_, __) => ContentCard(),
            separatorBuilder: (_, __) => 16.ph,
          ),
        ],
      ),
    ),
  );
 }
}
```